

**Name: Sarvesh Mishra**  
**Roll No: 2401330100319**  
**Branch: CSE**  
**Section: E**  
**Workshop Lab Number: 012**

## **Question 1**

### Student.java

```
package com.workshop.student.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Entity
@Table(name = "students")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String rollNumber;

    private String department;

    private Double marks;
}
```

### StudentRepository.java

```
package com.workshop.student.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.workshop.student.model.Student;
```

```
public interface StudentRepository extends JpaRepository<Student, Long> {
}
```

## [StudentController.java](#)

```
package com.workshop.student.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.workshop.student.exception.StudentNotFoundException;
import com.workshop.student.model.Student;
import com.workshop.student.repository.StudentRepository;

@RestController
@RequestMapping("/api/students")
public class StudentController {
    private final StudentRepository studentRepository;

    public StudentController(StudentRepository studentRepository) {
        this.studentRepository = studentRepository;
    }

    @PostMapping
    public ResponseEntity<Student> saveStudent(@RequestBody Student student) {
        Student savedStudent = studentRepository.save(student);
        return ResponseEntity.status(HttpStatus.CREATED).body(savedStudent);
    }

    @GetMapping
    public ResponseEntity<List<Student>> getAllStudents() {
        return ResponseEntity.ok(studentRepository.findAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getStudentById(@PathVariable Long id) {
        Student student = studentRepository.findById(id)
            .orElseThrow(() -> new StudentNotFoundException("Student not found with id " + id));
        return ResponseEntity.ok(student);
    }
}
```

## [StudentNotFoundException.java](#)

```
package com.workshop.student.exception;

public class StudentNotFoundException extends RuntimeException {
```

```

        public StudentNotFoundException(String message) {
            super(message);
        }
    }
}

```

## [GlobalExceptionHandler.java](#)

```

package com.workshop.student.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(StudentNotFoundException.class)
    public ResponseEntity<ApiError> handleStudentNotFound(StudentNotFoundException
exception) {
        ApiError apiError = new ApiError(HttpStatus.NOT_FOUND.value(),
exception.getMessage());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(apiError);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError>
handleValidationError(MethodArgumentNotValidException exception) {
        ApiError apiError = new ApiError(HttpStatus.BAD_REQUEST.value(), "Invalid
student details provided.");
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(apiError);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> handleGenericError(Exception exception) {
        ApiError apiError = new ApiError(HttpStatus.INTERNAL_SERVER_ERROR.value(),
"Something went wrong: " + exception.getMessage());
        return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(apiError);
    }
}

```

## [application.properties](#)

```

server.port=8083

spring.datasource.url=jdbc:h2:mem:studentdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

```

```
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

## Output

### POST /api/students

```
C:\>curl.exe -X POST "http://localhost:8083/api/students" -H "Content-Type: application/json" -d "{\"name\":\"Rohan Verma\",\"rollNumber\":\"ME105\",\"department\":\"Mechanical\",\"marks\":76.5}"
{"id":3,"name":"Rohan Verma","rollNumber":"ME105","department":"Mechanical","marks":76.5}
```

### GET /api/students

```
C:\>curl.exe "http://localhost:8083/api/students"
[{"id":1,"name":"Aarav Sharma","rollNumber":"CS101","department":"Computer Science","marks":88.5}, {"id":2,"name":"Ananya Gupta","rollNumber":"IT102","department":"Information Technology","marks":91.2}, {"id":3,"name":"Rohan Verma","rollNumber":"ME105","department":"Mechanical","marks":76.5}]
```

### GET /api/students/{id}

```
C:\>curl.exe "http://localhost:8083/api/students/1"
{"id":1,"name":"Aarav Sharma","rollNumber":"CS101","department":"Computer Science","marks":88.5}
```

### GET /api/students/99

```
C:\>curl.exe "http://localhost:8083/api/students/99"
{"timestamp":"2026-07-23T14:54:45.7158559","status":404,"message":"Student not found with id 99"}
```

## **Question 2**

### [Product.java](#)

```
package com.workshop.product.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    private String category;

    @Column(nullable = false)
    private Double price;

    private Integer quantity;

    public Product() {
    }

    public Product(String name, String category, Double price, Integer quantity) {
        this.name = name;
        this.category = category;
        this.price = price;
        this.quantity = quantity;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCategory() {
```

```

        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }

    public Double getPrice() {
        return price;
    }

    public void setPrice(Double price) {
        this.price = price;
    }

    public Integer getQuantity() {
        return quantity;
    }

    public void setQuantity(Integer quantity) {
        this.quantity = quantity;
    }
}

```

### [ProductRepository.java](#)

```

package com.workshop.product.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.workshop.product.model.Product;

public interface ProductRepository extends JpaRepository<Product, Long> {
}

```

### [ProductService.java](#)

```

package com.workshop.product.service;

import java.util.List;

import org.springframework.stereotype.Service;

import com.workshop.product.exception.ProductNotFoundException;
import com.workshop.product.model.Product;
import com.workshop.product.repository.ProductRepository;

@Service
public class ProductService {
    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public Product createProduct(Product product) {

```

```

        return productRepository.save(product);
    }

    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }

    public Product getProductById(Long id) {
        return productRepository.findById(id)
            .orElseThrow(() -> new ProductNotFoundException("Product not found
with id " + id));
    }

    public Product updateProduct(Long id, Product updatedProduct) {
        Product existingProduct = getProductById(id);
        existingProduct.setName(updatedProduct.getName());
        existingProduct.setCategory(updatedProduct.getCategory());
        existingProduct.setPrice(updatedProduct.getPrice());
        existingProduct.setQuantity(updatedProduct.getQuantity());
        return productRepository.save(existingProduct);
    }

    public void deleteProduct(Long id) {
        Product existingProduct = getProductById(id);
        productRepository.delete(existingProduct);
    }
}

```

## [ProductController.java](#)

```

package com.workshop.product.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.workshop.product.model.Product;
import com.workshop.product.service.ProductService;

@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @PostMapping

```

```

    public ResponseEntity<Product> createProduct(@RequestBody Product product) {
        Product savedProduct = productService.createProduct(product);
        return ResponseEntity.status(HttpStatus.CREATED).body(savedProduct);
    }

    @GetMapping
    public ResponseEntity<List<Product>> getAllProducts() {
        return ResponseEntity.ok(productService.getAllProducts());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(@PathVariable Long id) {
        return ResponseEntity.ok(productService.getProductById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Product> updateProduct(@PathVariable Long id,
    @RequestBody Product product) {
        return ResponseEntity.ok(productService.updateProduct(id, product));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<String> deleteProduct(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.ok("Product with id " + id + " deleted successfully.");
    }
}

```

### [ProductNotFoundException.java](#)

```

package com.workshop.product.exception;

public class ProductNotFoundException extends RuntimeException {
    public ProductNotFoundException(String message) {
        super(message);
    }
}

```

### [GlobalExceptionHandler.java](#)

```

package com.workshop.product.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
    public ResponseEntity<ApiError> handleProductNotFound(ProductNotFoundException
exception) {
        ApiError apiError = new ApiError(HttpStatus.NOT_FOUND.value(),
exception.getMessage());
    }
}

```



```

        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(apiError);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> handleGenericError(Exception exception) {
        ApiError apiError = new ApiError(HttpStatus.INTERNAL_SERVER_ERROR.value(),
"Something went wrong: " + exception.getMessage());
        return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(apiError);
    }
}

```

## [application.properties](#)

server.port=8084

spring.datasource.url=jdbc:mysql://localhost:3306/product\_crud\_app?createDatabaseIfNotExist=true&useSSL=false&serverTimezone=UTC

spring.datasource.username=root

spring.datasource.password=

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

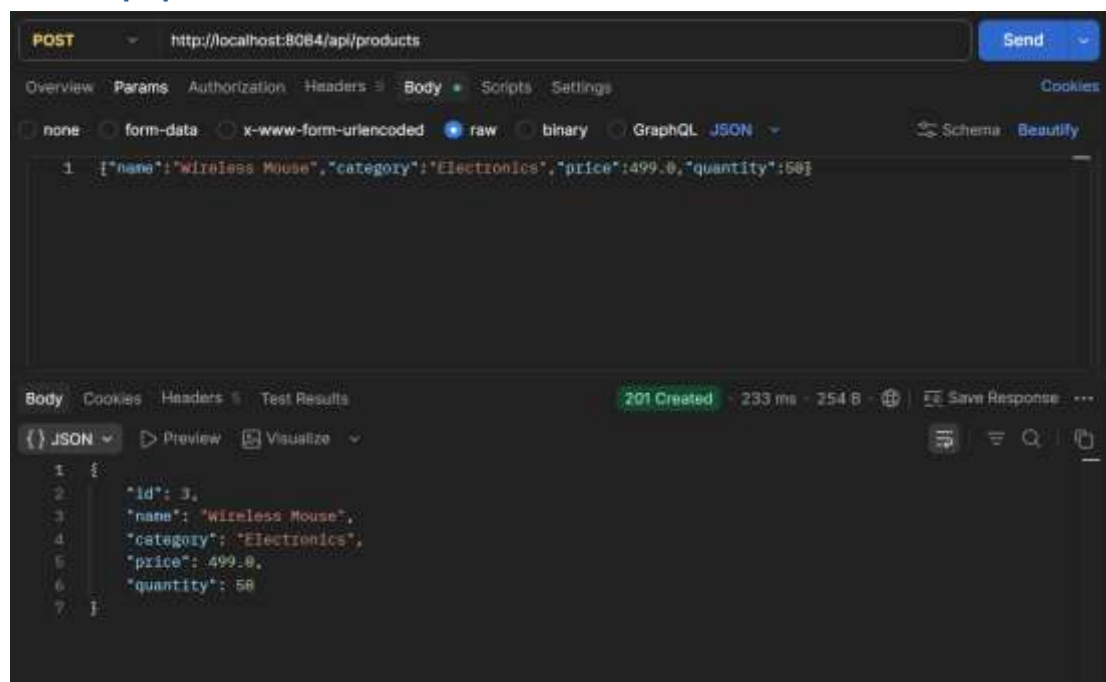
spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.open-in-view=false

## [Output](#)

### [POST /api/products](#)



## GET api/products

GET  Send

Overview Params Authorization Headers 7 Body Scripts Settings Cookies

### New Request

[View complete documentation →](#)

Add request description...

Body Cookies Headers 5 Test Results 200 OK · 83 ms · 341 B · [Save Response](#) · [...](#)

{} JSON Preview Visualize ...

```
1 [
2   {
3     "id": 1,
4     "name": "Wireless Mouse Pro",
5     "category": "Electronics",
6     "price": 599.8,
7     "quantity": 45
8   },
9   {
10    "id": 3,
11    "name": "Wireless Mouse",
12    "category": "Electronics",
13    "price": 499.8,
14    "quantity": 58
15  }
16 ]
```

## GET /api/products/1

PUT  Send

Overview Params Authorization Headers 9 Body 1 Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema: Beauty

```
1 [{"name": "Wireless Mouse Pro", "category": "Electronics", "price": 599.8, "quantity": 45}]
```

Body Cookies Headers 5 Test Results 200 OK · 67 ms · 253 B · [Save Response](#) · [...](#)

{} JSON Preview Visualize ...

```
1 {
2   "id": 1,
3   "name": "Wireless Mouse Pro",
4   "category": "Electronics",
5   "price": 599.8,
6   "quantity": 45
7 }
```

## [DELETE /api/products/1](#)

DELETE

http://localhost:8084/api/products/1

Send

OverviewParamsAuthorizationHeaders 7BodyScriptsSettingsCookies

# New Request

[View complete documentation →](#)

Add request description...

Body

200 OK • 46 ms • 203 B • [Save Response](#)

RawPreviewVisualize

1 Product with id 1 deleted successfully.

## [GET /api/products/1](#)

GET

http://localhost:8084/api/products/1

Send

OverviewParamsAuthorizationHeaders 7BodyScriptsSettingsCookies

# New Request

[View complete documentation →](#)

Add request description...

Body

404 Not Found • 31 ms • 267 B • [Save Response](#)

{ } JSONPreviewDebug with AI

```
1 {
2   "timestamp": "2026-07-23T15:20:01.8830029",
3   "status": 404,
4   "message": "Product not found with id 1"
5 }
```

## **Question 3**

### [Book.java](#)

```
package com.workshop.book.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "books")
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(nullable = false)
    private String author;

    @Column(nullable = false, unique = true)
    private String isbn;

    private Double price;

    public Book() {
    }

    public Book(String title, String author, String isbn, Double price) {
        this.title = title;
        this.author = author;
        this.isbn = isbn;
        this.price = price;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
```

```

        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public String getIsbn() {
        return isbn;
    }

    public void setIsbn(String isbn) {
        this.isbn = isbn;
    }

    public Double getPrice() {
        return price;
    }

    public void setPrice(Double price) {
        this.price = price;
    }
}

```

### [BookRepository.java](#)

```

package com.workshop.book.repository;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;

import com.workshop.book.model.Book;

public interface BookRepository extends JpaRepository<Book, Long> {
    Optional<Book> findByIsbn(String isbn);
}

```

### [BookService.java](#)

```

package com.workshop.book.service;

import java.util.List;

import org.springframework.stereotype.Service;

import com.workshop.book.exception.BookNotFoundException;
import com.workshop.book.exception.DuplicateIsbnException;
import com.workshop.book.model.Book;
import com.workshop.book.repository.BookRepository;

@Service
public class BookService {
    private final BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {

```

```

        this.bookRepository = bookRepository;
    }

    public Book createBook(Book book) {
        if (bookRepository.findByIsbn(book.getIsbn()).isPresent()) {
            throw new DuplicateIsbnException("A book with ISBN " + book.getIsbn()
+ " already exists.");
        }
        return bookRepository.save(book);
    }

    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    public Book getBookById(Long id) {
        return bookRepository.findById(id)
            .orElseThrow(() -> new BookNotFoundException("Book not found with
id " + id));
    }

    public Book updateBook(Long id, Book updatedBook) {
        Book existingBook = getBookById(id);
        existingBook.setTitle(updatedBook.getTitle());
        existingBook.setAuthor(updatedBook.getAuthor());
        existingBook.setIsbn(updatedBook.getIsbn());
        existingBook.setPrice(updatedBook.getPrice());
        return bookRepository.save(existingBook);
    }

    public void deleteBook(Long id) {
        Book existingBook = getBookById(id);
        bookRepository.delete(existingBook);
    }
}

```

## [BookController.java](#)

```

package com.workshop.book.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.workshop.book.model.Book;
import com.workshop.book.service.BookService;

```

```

@RestController
@RequestMapping("/api/books")
public class BookController {
    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @PostMapping
    public ResponseEntity<Book> createBook(@RequestBody Book book) {
        Book savedBook = bookService.createBook(book);
        return ResponseEntity.status(HttpStatus.CREATED).body(savedBook);
    }

    @GetMapping
    public ResponseEntity<List<Book>> getAllBooks() {
        return ResponseEntity.ok(bookService.getAllBooks());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Book> getBookById(@PathVariable Long id) {
        return ResponseEntity.ok(bookService.getBookById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Book> updateBook(@PathVariable Long id, @RequestBody
Book book) {
        return ResponseEntity.ok(bookService.updateBook(id, book));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<String> deleteBook(@PathVariable Long id) {
        bookService.deleteBook(id);
        return ResponseEntity.ok("Book with id " + id + " deleted successfully.");
    }
}

```

### [BookNotFoundException.java](#)

```

package com.workshop.book.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;

```

```

import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.workshop.book.model.Book;
import com.workshop.book.service.BookService;

@RestController
@RequestMapping("/api/books")
public class BookController {
    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @PostMapping
    public ResponseEntity<Book> createBook(@RequestBody Book book) {
        Book savedBook = bookService.createBook(book);
        return ResponseEntity.status(HttpStatus.CREATED).body(savedBook);
    }

    @GetMapping
    public ResponseEntity<List<Book>> getAllBooks() {
        return ResponseEntity.ok(bookService.getAllBooks());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Book> getBookById(@PathVariable Long id) {
        return ResponseEntity.ok(bookService.getBookById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Book> updateBook(@PathVariable Long id, @RequestBody
Book book) {
        return ResponseEntity.ok(bookService.updateBook(id, book));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<String> deleteBook(@PathVariable Long id) {
        bookService.deleteBook(id);
        return ResponseEntity.ok("Book with id " + id + " deleted successfully.");
    }
}

```

## [DuplicateIsbnException.java](#)

```

package com.workshop.book.exception;

```



```

public class DuplicateIsbnException extends RuntimeException {
    public DuplicateIsbnException(String message) {
        super(message);
    }
}

```

### [GlobalExceptionHandler.java](#)

```

package com.workshop.book.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ResponseBody;

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BookNotFoundException.class)
    @ResponseBody
    public ResponseEntity<ApiError> handleBookNotFound(BookNotFoundException
exception) {
        ApiError apiError = new ApiError(HttpStatus.NOT_FOUND.value(),
exception.getMessage());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(apiError);
    }

    @ExceptionHandler(DuplicateIsbnException.class)
    @ResponseBody
    public ResponseEntity<ApiError> handleDuplicateIsbn(DuplicateIsbnException
exception) {
        ApiError apiError = new ApiError(HttpStatus.CONFLICT.value(),
exception.getMessage());
        return ResponseEntity.status(HttpStatus.CONFLICT).body(apiError);
    }
}

```

```

    }

    @ExceptionHandler(Exception.class)
    @ResponseBody
    public ResponseEntity<ApiError> handleGenericError(Exception exception) {
        ApiError apiError = new ApiError(HttpStatus.INTERNAL_SERVER_ERROR.value(),
            "Something went wrong: " + exception.getMessage());

        return
        ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(apiError);
    }
}

```

## Output

### POST /api/books

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8085/api/books
- Send Button:** Send
- Overview Tab:** Params, Authorization, Headers, **Body** (selected), Scripts, Settings, Cookies
- Body Tab:** raw (selected), JSON (selected), Schema, Beautify
- Request Body (raw):**

```
1 {"title":"Clean Code 2","author":"Robert Martin","isbn":"9780132350885",
  "price":650.0}
```
- Response:**
  - Status:** 201 Created
  - Time:** 22 ms
  - Size:** 262 B
  - Actions:** Save Response, ...
  - Body Tab:** {} JSON (selected), Preview, Visualize
  - Response Body (JSON):**

```
1 {
2   "id": 3,
3   "title": "Clean Code 2",
4   "author": "Robert Martin",
5   "isbn": "9780132350885",
6   "price": 650.0
7 }
```

## [PUT /api/books/2](#)

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** http://localhost:8085/api/books/2
- Buttons:** Send, Overview, Params, Authorization, Headers 9, Body (selected), Scripts, Settings, Cookies
- Body Tab:** raw (selected), JSON (selected), Schema, Beautify
- Request Body (JSON):**

```
1  {"title": "Clean Code Second Edition", "author": "Robert Martin",  
   "isbn": "9780132350884", "price": 750.0}
```
- Response:** 200 OK, 32 ms, 270 B, Save Response
- Response Body (JSON):**

```
1  {  
2    "id": 2,  
3    "title": "Clean Code Second Edition",  
4    "author": "Robert Martin",  
5    "isbn": "9780132350884",  
6    "price": 750.0  
7  }
```

## [GET /api/books/2](#)

The screenshot shows a REST client interface with the following details:

- Method:** DELETE
- URL:** http://localhost:8085/api/books/2
- Buttons:** Send, Overview (selected), Params, Authorization, Headers 7, Body, Scripts, Settings, Cookies
- Header:** New Request, View complete documentation →
- Response:** 200 OK, 29 ms, 200 B, Save Response
- Response Body (Raw):**

```
1  Book with id 2 deleted successfully.
```

## [GET /api/books/2](#)

The screenshot shows a REST client interface with a GET request to `http://localhost:8085/api/books/2`. The response is a 404 Not Found status, indicating the resource does not exist. The response body is in JSON format, showing a timestamp, status, and a message.

**GET** `http://localhost:8085/api/books/2` **Send**

Overview Params Authorization Headers 7 Body Scripts Settings Cookies

### New Request

[View complete documentation →](#)

Add request description...

Body **404 Not Found** • 23 ms • 264 B • [Save Response](#) ...

**JSON** Preview Debug with AI

```
1 {
2   "timestamp": "2026-07-23T15:43:49.6918926",
3   "status": 404,
4   "message": "Book not found with id 2"
5 }
```

## [POST /api/books](#)

The screenshot shows a REST client interface with a POST request to `http://localhost:8085/api/books`. The request body is a JSON object representing a book. The response is a 201 Created status, indicating the resource was successfully created. The response body is in JSON format, showing the created book's details.

**POST** `http://localhost:8085/api/books` **Send**

Overview Params Authorization Headers 9 **Body** Scripts Settings Cookies

raw **JSON** Schema Beautify

```
1 {"title": "Clean Code", "author": "Robert Martin", "isbn": "9780132350888",
   "price": 650.0}
```

Body **201 Created** • 24 ms • 260 B • [Save Response](#) ...

**JSON** Preview Visualize

```
1 {
2   "id": 4,
3   "title": "Clean Code",
4   "author": "Robert Martin",
5   "isbn": "9780132350888",
6   "price": 650.0
7 }
```

## [POST /api/books](#)

The screenshot shows a REST client interface with a POST request to `http://localhost:8085/api/books`. The request body is a JSON object: `{"title": "Clean Code", "author": "Robert Martin", "isbn": "9780132350884", "price": 650.0}`. The response is a `409 Conflict` status with a timestamp, status code, and a message: `"A book with ISBN 9780132350884 already exists."`.

**POST** `http://localhost:8085/api/books` **Send**

Overview Params Authorization Headers 9 **Body** Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {"title": "Clean Code", "author": "Robert Martin", "isbn": "9780132350884",  
  "price": 650.0}
```

Body **409 Conflict** • 13 ms • 285 B • Save Response ...

{ } JSON Preview Debug with AI

```
1 {  
2   "timestamp": "2026-07-23T15:48:00.9107987",  
3   "status": 409,  
4   "message": "A book with ISBN 9780132350884 already exists."  
5 }
```

## **Question 4**

### Course.java

```
package com.workshop.course.model;

import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import lombok.ToString;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@ToString
public class Course {
    private int id;
    private String courseCode;
    private String courseName;
    private String instructor;
    private int credits;
}
```

### Course.java

```
package com.workshop.course.model;

import org.springframework.stereotype.Component;

@Component
public class Course {
    private int courseId = 501;
    private String courseName = "Advanced Java";
    private String facultyName = "Dr. Sharma";

    public int getCourseId() {
        return courseId;
    }

    public String getCourseName() {
        return courseName;
    }

    public String getFacultyName() {
        return facultyName;
    }
}
```

### CourseService.java

```
package com.workshop.course.service;

import java.util.List;
```

```

import org.springframework.stereotype.Service;

import com.workshop.course.exception.CourseNotFoundException;
import com.workshop.course.model.Course;

@Service
public class CourseService {
    private final List<Course> courses = List.of(
        new Course(1, "CS301", "Advanced Java", "Dr. Neha Sharma", 4),
        new Course(2, "CS302", "Data Structures", "Prof. Amit Rao", 3),
        new Course(3, "CS303", "Database Management Systems", "Dr. Kavita
Joshi", 3));

    public List<Course> getAllCourses() {
        return courses;
    }

    public Course getCourseById(int id) {
        return courses.stream()
            .filter(course -> course.getId() == id)
            .findFirst()
            .orElseThrow(() -> new CourseNotFoundException("Course not found
with id " + id));
    }
}

```

## [CourseController.java](#)

```

package com.workshop.course.controller;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.workshop.course.model.Course;
import com.workshop.course.service.CourseService;

@RestController
@RequestMapping("/api/courses")
public class CourseController {
    private final CourseService courseService;

    public CourseController(CourseService courseService) {
        this.courseService = courseService;
    }

    @GetMapping
    public ResponseEntity<List<Course>> getAllCourses() {
        return ResponseEntity.ok(courseService.getAllCourses());
    }
}

```

```

@GetMapping("/{id}")
public ResponseEntity<Course> getCourseById(@PathVariable int id) {
    return ResponseEntity.ok(courseService.getCourseById(id));
}

@GetMapping(value = "{id}/view", produces = MediaType.TEXT_HTML_VALUE)
public ResponseEntity<String> viewCourse(@PathVariable int id) {
    Course course = courseService.getCourseById(id);
    String html = ""
        <html>
        <body style="font-family:Arial;padding:20px;">
        <h2>Course Details</h2>
        <table border="1" cellpadding="8" cellspacing="0">
            <tr><th>Course Code</th><td>%s</td></tr>
            <tr><th>Course Name</th><td>%s</td></tr>
            <tr><th>Instructor</th><td>%s</td></tr>
            <tr><th>Credits</th><td>%d</td></tr>
        </table>
        </body>
        </html>
        """.formatted(course.getCourseCode(), course.getCourseName(),
course.getInstructor(), course.getCredits());
    return ResponseEntity.status(HttpStatus.OK).body(html);
}
}

```

## [CourseInfoAppApplication.java](#)

```

package com.workshop.course;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CourseInfoAppApplication {
    public static void main(String[] args) {
        SpringApplication.run(CourseInfoAppApplication.class, args);
    }
}

```

## [CourseInfoAppApplication.java](#)

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Course Info App</title>
    <style>
        body { font-family: Arial, sans-serif; padding: 30px; background: #f4f6f8; }
        h1 { color: #2c3e50; }
        a { display: inline-block; margin: 6px 0; color: #2563eb; }
        .card { background: white; padding: 20px; border-radius: 8px; max-width: 480px; box-shadow: 0 1px 4px rgba(0,0,0,0.1); }
    </style>
</head>
<body>

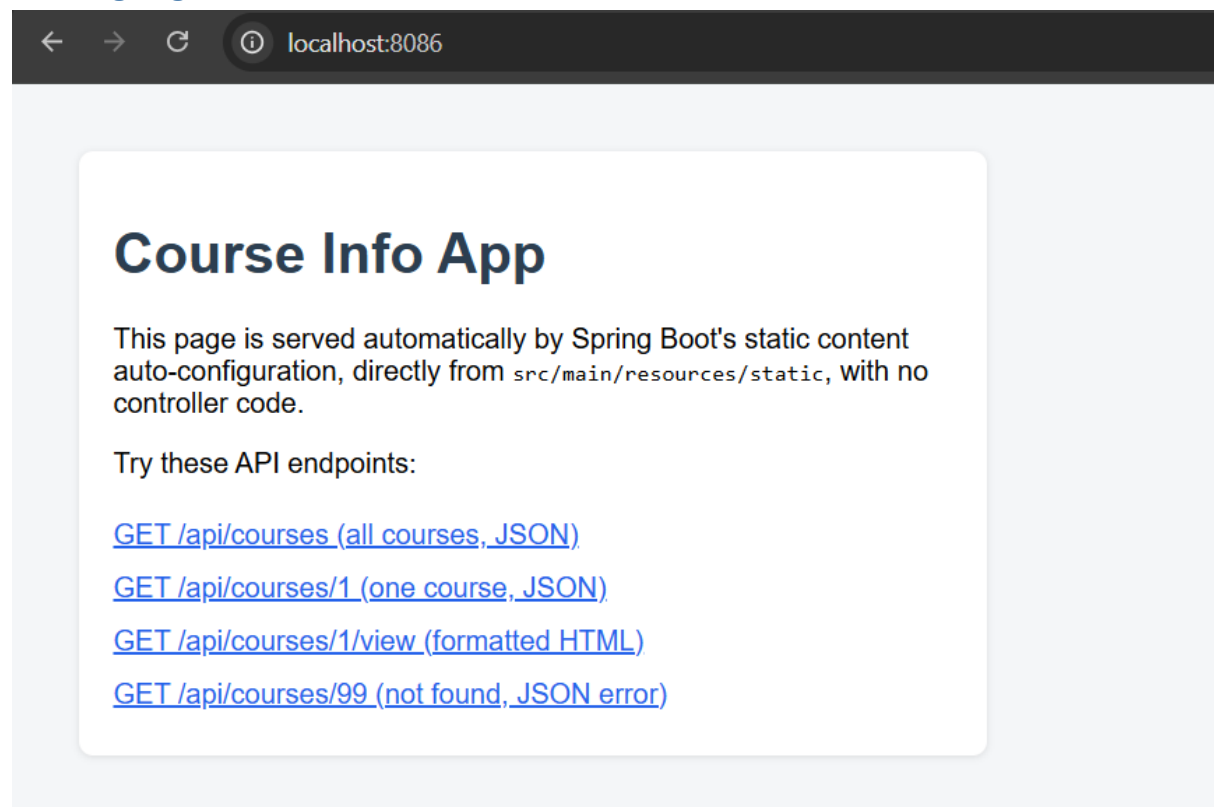
```



```
<div class="card">
  <h1>Course Info App</h1>
  <p>This page is served automatically by Spring Boot's static content auto-
configuration, directly from <code>src/main/resources/static</code>, with no
controller code.</p>
  <p>Try these API endpoints:</p>
  <a href="/api/courses">GET /api/courses (all courses, JSON)</a><br>
  <a href="/api/courses/1">GET /api/courses/1 (one course, JSON)</a><br>
  <a href="/api/courses/1/view">GET /api/courses/1/view (formatted
HTML)</a><br>
  <a href="/api/courses/99">GET /api/courses/99 (not found, JSON error)</a>
</div>
</body>
</html>
```

## [Output](#)

### [Landing Page](#)



</api/courses>

```
[
  {
    "id": 1,
    "courseCode": "CS301",
    "courseName": "Advanced Java",
    "instructor": "Dr. Neha Sharma",
    "credits": 4
  },
  {
    "id": 2,
    "courseCode": "CS302",
    "courseName": "Data Structures",
    "instructor": "Prof. Amit Rao",
    "credits": 3
  },
  {
    "id": 3,
    "courseCode": "CS303",
    "courseName": "Database Management Systems",
    "instructor": "Dr. Kavita Joshi",
    "credits": 3
  }
]
```

</api/courses/1>

```
{
  "id": 1,
  "courseCode": "CS301",
  "courseName": "Advanced Java",
  "instructor": "Dr. Neha Sharma",
  "credits": 4
}
```

</api/courses/1/view>

## Course Details

|                    |                 |
|--------------------|-----------------|
| <b>Course Code</b> | CS301           |
| <b>Course Name</b> | Advanced Java   |
| <b>Instructor</b>  | Dr. Neha Sharma |
| <b>Credits</b>     | 4               |

</api/courses/99>

```
{
  "timestamp": "2026-07-23T15:53:35.893321",
  "status": 404,
  "message": "Course not found with id 99"
}
```